

- Description

This module manages a smart boiler system, ensuring that:

1. **The water temperature inside the boiler is continuously monitored to maintain optimal heating efficiency.**
2. **If the temperature drops below the minimum threshold, the heating system is activated.**
3. **If the temperature rises above the maximum threshold, the heating system is deactivated to prevent overheating.**
4. **A log of all temperature readings, heating system activations, and shutdowns is maintained for energy optimization.**

This system helps improve heating efficiency, prevent overheating, and optimize energy consumption in water heating applications.

- Specification

module SmartBoilerTemperatureControlEx

var currentTemperature, maxTemperature, minTemperature: real

var heatingActive: boolean

process updateTemperature(changeAmount: real)

functional scenario form:

currentTemperature <= maxTemperature && Water temperature maintained.

||

currentTemperature < minTemperature && heatingActive = true, Heating system activated.

||

currentTemperature > maxTemperature && heatingActive = false, Heating system deactivated.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.List;
```

```

import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c131_SmartBoilerSystemEx
 * Domain: Smart boiler with heating control, safety, and monitoring.
 *
 * Goals:
 * - Preserve the original decision inside ONE core function:
 *   public synchronized void updateTemperature(double changeAmount)
 *   Branches under test:
 *   (1) currentTemperature += changeAmount;
 *   (2) if (currentTemperature < minTemperature) heatingActive = t
rue;
 *   else if (currentTemperature > maxTemperature) heatingActive
= false;
 *   (3) Print/log statements retained exactly.
 *
 * Features:
 * - Sensors: water temp, pressure, flow rate, energy meter, limescale
index
 * - Controller: HeaterController (clamps power/ramp), NOT making the
decision
 * - Logger + Alerts (file-backed)
 * - Swing GUI: status, thresholds, heater power limits, simulation co
ntrols,
 *   alerts, and log tail
 * - Simulation engine: ambient exchange, heat loss, tap demand, refil
l
 * - Replay engine: parse "Env step: ..." from log and replay
 * - CLI harness: VT/FT/Z3/UVT with "comment 3 lines to run only one"
 *
 * Compile: javac code/c131_SmartBoilerSystemEx.java
 * Run GUI: java code.c131_SmartBoilerSystemEx
 * Run CLI: java code.c131_SmartBoilerSystemEx --cli
 * =====
 */
/* ===== Utilities ===== */
class BStr {
    static String fmt2(double v) { return new DecimalFormat("0.00").for
mat(v); }
    static String fmt3(double v) { return new DecimalFormat("0.000").fo
rmat(v); }

```

```

        static String now() { return LocalDateTime.now().format(DateTimeForm
atter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
    }
    /* ===== Logger ===== */
    class BDataLogger {
        private final File file;
        private final ConcurrentLinkedQueue<String> inMemory = new Concurr
entLinkedQueue<>();
        private final int cap;
        BDataLogger(String path, int inMemoryCap) {
            this.file = new File(path);
            this.cap = Math.max(200, inMemoryCap);
            try {
                if (!file.exists()) {
                    File parent = file.getParentFile();
                    if (parent != null) parent.mkdirs();
                    file.createNewFile();
                }
            } catch (IOException e) {
                System.err.println("[Logger] Create file failed: " + e.getM
essage());
            }
        }
        synchronized void log(String message) {
            String line = "[" + BStr.now() + "] " + message;
            inMemory.add(line);
            while (inMemory.size() > cap) inMemory.poll();
            try (FileOutputStream fos = new FileOutputStream(file, true);
                OutputStreamWriter osw = new OutputStreamWriter(fos, Stand
ardCharsets.UTF_8);
                BufferedWriter bw = new BufferedWriter(osw)) {
                bw.write(line);
                bw.newLine();
            } catch (IOException e) {
                System.err.println("[Logger] Write failed: " + e.getMessage
());
            }
        }
        List<String> recent(int n) {
            ArrayList<String> list = new ArrayList<>(inMemory);
            if (list.size() <= n) return list;
            return list.subList(list.size() - n, list.size());
        }
        File getFile() { return file; }
    }

```

```

}
/* ===== Alerts ===== */
class BAlertSystem {
    enum Severity { INFO, WARNING, CRITICAL }
    static class Alert {
        final Severity severity;
        final String message;
        final LocalDateTime time = LocalDateTime.now();
        Alert(Severity s, String m) { this.severity = s; this.message =
m; }
        @Override public String toString() { return "[" + time + "]" +
severity + " " + message; }
    }
    private final List<Alert> alerts = new ArrayList<>();
    private final BDataLogger logger;
    BAlertSystem(BDataLogger logger) { this.logger = logger; }
    void push(Severity s, String msg) {
        Alert a = new Alert(s, msg);
        alerts.add(a);
        logger.log("ALERT " + s + " :: " + msg);
        System.out.println(a);
    }
    List<Alert> recent(int n) {
        if (alerts.size() <= n) return new ArrayList<>(alerts);
        return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
    }
}
/* ===== Sensors ===== */
abstract class BSensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    BSensor(String name) { this.name = name; }
    abstract void update(BoilerContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}
class BoilerContext {
    public double currentTemperature; // °C

```

```

    public double minTemperature;    // °C
    public double maxTemperature;    // °C
    public boolean heatingActive;
    public double pressureBar;       // bar
    public double flowRateLps;       // L/s
    public double ambientTemperature; // °C
    public double heaterPowerKW;     // kW
    public double limescaleIdx;       // 0..1
    public double energyKWh;          // cumulative
    BoilerContext copy() {
        BoilerContext c = new BoilerContext();
        c.currentTemperature = currentTemperature;
        c.minTemperature = minTemperature;
        c.maxTemperature = maxTemperature;
        c.heatingActive = heatingActive;
        c.pressureBar = pressureBar;
        c.flowRateLps = flowRateLps;
        c.ambientTemperature = ambientTemperature;
        c.heaterPowerKW = heaterPowerKW;
        c.limescaleIdx = limescaleIdx;
        c.energyKWh = energyKWh;
        return c;
    }
}

class WaterTempSensor extends BSensor {
    WaterTempSensor() { super("WaterTempSensor"); }
    @Override
    void update(BoilerContext ctx) {
        value = ctx.currentTemperature + (rnd.nextDouble() - 0.5) * 0.4
;
        ok = value > 0 && value < 110;
        status = ok ? (value > 85 ? "HOT" : "OK") : "OUT_OF_RANGE";
    }
}

class PressureSensor extends BSensor {
    PressureSensor() { super("PressureSensor"); }
    @Override
    void update(BoilerContext ctx) {
        value = Math.max(0, ctx.pressureBar + (rnd.nextDouble() - 0.5)
* 0.05);
        ok = value >= 0 && value < 10;
        status = ok ? (value > 3 ? "HIGH" : "OK") : "BROKEN";
    }
}

```

```

class FlowRateSensor extends BSensor {
    FlowRateSensor() { super("FlowRateSensor"); }
    @Override
    void update(BoilerContext ctx) {
        value = Math.max(0, ctx.flowRateLps + (rnd.nextDouble() - 0.5)
* 0.05);
        ok = value >= 0 && value < 30;
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class EnergyMeterSensor extends BSensor {
    EnergyMeterSensor() { super("EnergyMeterSensor"); }
    @Override
    void update(BoilerContext ctx) {
        value = Math.max(0, ctx.energyKWh + (rnd.nextDouble() - 0.5) *
0.01);
        ok = value >= 0 && value < 1e6;
        status = ok ? "OK" : "BROKEN";
    }
}

class LimescaleSensor extends BSensor {
    LimescaleSensor() { super("LimescaleSensor"); }
    @Override
    void update(BoilerContext ctx) {
        value = Math.max(0, Math.min(1.0, ctx.limescaleIdx + (rnd.nextD
ouble() - 0.5) * 0.01));
        ok = value >= 0 && value <= 1.0;
        status = ok ? (value > 0.7 ? "HIGH" : (value > 0.4 ? "MEDIUM" :
"LOW")) : "BROKEN";
    }
}

/* ===== Heater Controller (physical helper) =====
===== */
class HeaterController {
    private double maxPowerKW = 12.0; // kW physical limit
    private double rampKW = 1.0;     // kW per step
    private double currentKW = 0.0;
    double setpoint(double desiredKW) {
        desiredKW = Math.max(0, Math.min(maxPowerKW, desiredKW));
        // ramp limit
        double d = desiredKW - currentKW;
        if (d > rampKW) d = rampKW;
        if (d < -rampKW) d = -rampKW;
        currentKW += d;
    }
}

```

```

        return currentKW;
    }
    double getCurrentKW() { return currentKW; }
    void setMaxPowerKW(double kw) { maxPowerKW = Math.max(1.0, Math.min(50.0, kw)); }
    void setRampKW(double r) { rampKW = Math.max(0.2, Math.min(5.0, r)); }
; }
}
/* ===== Core Domain (SmartBoiler) =====
===== */
class SmartBoiler {
    private final BDataLogger logger;
    private final BAlertSystem alert;
    // thresholds
    private double maxTemperature; // °C
    private double minTemperature; // °C
    // state
    private double currentTemperature; // °C
    private boolean heatingActive;
    // environment/process
    private double pressureBar = 1.5; // bar
    private double flowRateLps = 0.0; // L/s (tap demand)
    private double ambientTemperature = 22.0; // °C
    private double limescaleIdx = 0.2; // 0..1 (higher worse)
    private double energyKWh = 0.0; // cumulative energy
    private double tankCapacityL = 120.0; // liters (toy)
    private double heaterEfficiency = 0.88; // efficiency factor
    // controller
    private final HeaterController heater = new HeaterController();
    // sensors
    private final WaterTempSensor tempSensor = new WaterTempSensor();
    private final PressureSensor pressureSensor = new PressureSensor();
    private final FlowRateSensor flowSensor = new FlowRateSensor();
    private final EnergyMeterSensor energySensor = new EnergyMeterSensor();
    SmartBoiler(double maxTemperature, double minTemperature, double initialTemperature,
        BDataLogger logger, BAlertSystem alert) {
        this.maxTemperature = maxTemperature;
        this.minTemperature = minTemperature;
        this.currentTemperature = initialTemperature;
        this.heatingActive = false;
    }
}

```

```

        this.logger = logger;
        this.alert = alert;
        logger.log("SmartBoiler initialized. Temp=" + BStr.fmt2(current
Temperature) + "°C"
                + " (min=" + BStr.fmt2(minTemperature) + ", max=" + BSt
r.fmt2(maxTemperature) + ")");
    }
    void addListener(Runnable r) { listeners.add(r); }
    private void emit() { for (Runnable r : listeners) try { r.run(); }
catch (Throwable ignored) {} }
    BoilerContext buildContext() {
        BoilerContext c = new BoilerContext();
        c.currentTemperature = currentTemperature;
        c.minTemperature = minTemperature;
        c.maxTemperature = maxTemperature;
        c.heatingActive = heatingActive;
        c.pressureBar = pressureBar;
        c.flowRateLps = flowRateLps;
        c.ambientTemperature = ambientTemperature;
        c.heaterPowerKW = heater.getCurrentKW();
        c.limescaleIdx = limescaleIdx;
        c.energyKWh = energyKWh;
        return c;
    }
    private void sampleSensors() {
        BoilerContext ctx = buildContext();
        tempSensor.update(ctx);
        pressureSensor.update(ctx);
        flowSensor.update(ctx);
        energySensor.update(ctx);
        limeSensor.update(ctx);
        // Alerts (safety envelopes)
        if (currentTemperature < minTemperature) {
            alert.push(BAlertSystem.Severity.INFO, "Below MIN temperatu
re (" + BStr.fmt2(currentTemperature) + " < " + BStr.fmt2(minTemperatur
e) + "°C)");
        }
        if (currentTemperature > maxTemperature) {
            alert.push(BAlertSystem.Severity.WARNING, "Above MAX tempera
ture (" + BStr.fmt2(currentTemperature) + " > " + BStr.fmt2(maxTempera
ture) + "°C)");
        }
        if (pressureSensor.getValue() > 3.5) {

```

```

        alert.push(BAlertSystem.Severity.CRITICAL, "High pressure:
" + BStr.fmt2(pressureSensor.getValue()) + " bar");
    }
    if (limeSensor.getValue() > 0.75) {
        alert.push(BAlertSystem.Severity.WARNING, "Limescale high:
idx=" + BStr.fmt2(limeSensor.getValue()));
    }
}
/* =====
* CORE FUNCTION: Keep the original logic in ONE function.
* Branches to cover:
*   - currentTemperature += changeAmount;
*   - if (< min) heatingActive = true;
*   else if (> max) heatingActive = false;
*   - Print/log statements retained.
* Extended tasks (controller/physics/sensors) happen around it
* but the decision remains exclusively here.
* ===== */
public synchronized void updateTemperature(double changeAmount) {
    // (1) Update temperature by external delta
    currentTemperature += changeAmount;
    // (2) Original decision Logic here (one place)
    if (currentTemperature < minTemperature) {
        heatingActive = true;
    } else if (currentTemperature > maxTemperature) {
        heatingActive = false;
    } // between: keep state
    // (3) Controller application (does not decide on/off)
    double desiredKW = heatingActive ? 10.0 : 0.0; // target heater
power
    double powerKW = heater.setpoint(desiredKW);
    // Approximate thermal effect to water temperature using power,
effect to make the system dynamic; core decision remains above.
    // This is a side-
    double seconds = 1.0; // assume per-call ~1 second in sim/CLI
    double joules = powerKW * 1000.0 * seconds * heaterEfficiency;
// kW -> kJ/s -> J
    double massKg = tankCapacityL * 1.0; // 1L ~ 1kg
    double cWater = 4184.0; // J/(kg·°C)
    double dT_up = joules / (massKg * cWater);
    // Heat loss proportional to difference with ambient, plus draw
by flow

```

```

        double loss = Math.max(0, (currentTemperature - ambientTemperature) * 0.002 * seconds);
        double draw = Math.max(0, flowRateLps * 0.1 * seconds); // tapping cools water slightly
        double newTemp = currentTemperature + dT_up - loss - draw;
        // Apply small clamp to physical bounds
        currentTemperature = Math.max(0, Math.min(110, newTemp));
        // Energy accounting
        energyKWh += (powerKW * seconds) / 3600.0;
        // (4) Print/log per original format
        String line = "Current Water Temperature: " + BStr.fmt2(currentTemperature) + "°C";
        logger.log(line);
        System.out.println(line);
        if (heatingActive) {
            System.out.println("Heating System Activated.");
        } else {
            System.out.println("Heating System Deactivated.");
        }
        // (5) Sensors & UI
        sampleSensors();
        emit();
    }
    /* ===== Simulation hooks ===== */
    synchronized void tickSeconds(double seconds) {
        // During idle ticks, we call updateTemperature with a tiny correction
        // so coverage still focuses on the core function.
        double passiveLoss = -
(currentTemperature - ambientTemperature) * 0.0008 * seconds;
        double tapEffect = -flowRateLps * 0.05 * seconds;
        updateTemperature(passiveLoss + tapEffect);
        // Pressure dynamics (toy)
        pressureBar += (heatingActive ? +0.01 : -0.008) * seconds;
        pressureBar = Math.max(0.8, Math.min(4.5, pressureBar));
        // Limescale grows faster at high temps
        limescaleIdx = Math.max(0, Math.min(1.0, limescaleIdx + (currentTemperature > 70 ? 0.0005 : 0.0001) * seconds));
    }
    synchronized void applyEnvironment(double dAmb, double dFlow, double demandDrawL, double refillL) {
        ambientTemperature += dAmb;
        flowRateLps = Math.max(0, flowRateLps + dFlow);
    }

```

```

        // Demand draw and refill slightly affect effective temperature
        (toy mix with ambient)
        if (demandDrawL > 0) {
            double mix = demandDrawL / Math.max(1.0, tankCapacityL);
            currentTemperature = (1 - mix) * currentTemperature + mix *
ambientTemperature;
        }
        if (refillL > 0) {
            double mix = refillL / Math.max(1.0, tankCapacityL);
            currentTemperature = (1 - mix) * currentTemperature + mix *
ambientTemperature;
        }
        logger.log(String.format(Locale.US,
            "Env step: dAmb=%.3f °C, dFlow=%.3f L/s, draw=%.2f L, re
fill=%.2f L | Amb=%.2f, Flow=%.2f",
            dAmb, dFlow, demandDrawL, refillL, ambientTemperature,
flowRateLps));
        emit();
    }
    /* ===== Maintenance/Config ===== */
    synchronized void calibrateTemperature(double observedC) {
        double old = currentTemperature;
        currentTemperature = Math.max(0, Math.min(110, observedC));
        logger.log("Calibration: temp " + BStr.fmt2(old) + " -> " + BSt
r.fmt2(currentTemperature) + " °C");
        sampleSensors();
        emit();
    }
    synchronized void setThresholds(double maxT, double minT) {
        double oldMax = maxTemperature, oldMin = minTemperature;
        maxTemperature = Math.max(minT + 1, maxT);
        minTemperature = Math.min(minT, maxTemperature - 1);
        logger.log("Thresholds changed: max " + BStr.fmt2(oldMax) + "->
" + BStr.fmt2(maxTemperature) +
            ", min " + BStr.fmt2(oldMin) + "->" + BStr.fmt2(minTemp
erature));
        sampleSensors();
        emit();
    }
    synchronized void setHeaterLimits(double maxKW, double rampKW) {
        heater.setMaxPowerKW(maxKW);
        heater.setRampKW(rampKW);
        logger.log("Heater limits set: maxKW=" + BStr.fmt2(maxKW) + ",
rampKW=" + BStr.fmt2(rampKW));
    }

```

```

    }
    /* ===== Getters for GUI/Tests ===== */
    double getCurrentTemperature() { return currentTemperature; }
    double getMinTemperature() { return minTemperature; }
    double getMaxTemperature() { return maxTemperature; }
    boolean isHeatingActive() { return heatingActive; }
    double getPressureBar() { return pressureBar; }
    double getFlowRateLps() { return flowRateLps; }
    double getAmbientTemperature() { return ambientTemperature; }
    double getLimescaleIdx() { return limescaleIdx; }
    double getEnergyKWh() { return energyKWh; }
    WaterTempSensor getTempSensor() { return tempSensor; }
    PressureSensor getPressureSensor() { return pressureSensor; }
    FlowRateSensor getFlowSensor() { return flowSensor; }
    EnergyMeterSensor getEnergySensor() { return energySensor; }
    LimescaleSensor getLimeSensor() { return limeSensor; }
}
/* ===== Simulation Engine =====
= */
class BoilerSimulationEngine implements Runnable {
    private final SmartBoiler boiler;
    private final BDataLogger logger;
    private final AtomicBoolean running = new AtomicBoolean(false);
    private Thread thread;
    private volatile double dAmb = +0.03;    // °C per tick
    private volatile double dFlow = +0.01;    // L/s per tick
    private volatile double drawL = 0.5;      // liters per tick
    private volatile double refillL = 0.5;    // liters per tick
    private volatile double tickSeconds = 1.0;
    private volatile long sleepMillis = 250L;
    private volatile boolean randomize = true;
    private final Random rnd = new Random();
    BoilerSimulationEngine(SmartBoiler boiler, BDataLogger logger) {
        this.boiler = boiler;
        this.logger = logger;
    }
    void start() {
        if (running.get()) return;
        running.set(true);
        thread = new Thread(this, "BoilerSimEngine");
        thread.setDaemon(true);
        thread.start();
        logger.log("Boiler simulation started.");
    }
}

```

```

    void stop() {
        running.set(false);
        if (thread != null) {
            try { thread.join(2000); } catch (InterruptedException ignored) {}
        }
        logger.log("Boiler simulation stopped.");
    }
    boolean isRunning() { return running.get(); }
    void setParams(double dAmb, double dFlow, double drawL, double refillL,
        double tickSeconds, long delayMs, boolean randomize)
    {
        this.dAmb = dAmb;
        this.dFlow = dFlow;
        this.drawL = drawL;
        this.refillL = refillL;
        this.tickSeconds = Math.max(0.1, tickSeconds);
        this.sleepMillis = Math.max(20, delayMs);
        this.randomize = randomize;

        logger.log(String.format(Locale.US,
            "Sim params updated: dAmb=%.3f, dFlow=%.3f, draw=%.2fL,
            refill=%.2fL, dt=%.2fs, delay=%dms, random=%s",
            dAmb, dFlow, drawL, refillL, this.tickSeconds, this.sleepMillis, String.valueOf(this.randomize)));
    }
    @Override
    public void run() {
        while (running.get()) {
            double a = dAmb, f = dFlow, draw = drawL, ref = refillL;
            if (randomize) {
                a += (rnd.nextDouble() - 0.5) * Math.max(0.2, Math.abs(dAmb) * 2.0);
                f += (rnd.nextDouble() - 0.5) * Math.max(0.05, Math.abs(dFlow) * 2.0);
                draw = Math.max(0, draw + (rnd.nextDouble() - 0.5) * Math.max(0.3, Math.abs(drawL)));
                ref = Math.max(0, ref + (rnd.nextDouble() - 0.5) * Math.max(0.3, Math.abs(refillL)));
            }
            boiler.applyEnvironment(a, f, draw, ref);
            boiler.tickSeconds(tickSeconds);
            try { Thread.sleep(sleepMillis); }

```

```

        catch (InterruptedException e) { break; }
    }
}

/* ===== Replay Engine ===== */
class BoilerReplayEngine {
    private final SmartBoiler boiler;
    private final BDataLogger logger;
    BoilerReplayEngine(SmartBoiler boiler, BDataLogger logger) {
        this.boiler = boiler;
        this.logger = logger;
    }
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
        logger.log("Replay started from: " + logFile.getAbsolutePath());
;
        try (BufferedReader br = new BufferedReader(new InputStreamReader(
            new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;

            while ((line = br.readLine()) != null) {
                if (!line.contains("Env step:")) continue;
                try {
                    int idx = line.indexOf("Env step:");
                    String seg = line.substring(idx);
                    // Env step: dAmb=..., dFlow=..., draw=..., refill=
... / ...

                    String[] parts = seg.split("\\|")[0].split(",");
                    double a = Double.parseDouble(parts[0].split("=")[1]
].replace("°C", "").trim());
                    double f = Double.parseDouble(parts[1].split("=")[1]
].replace("L/s", "").trim());
                    double d = Double.parseDouble(parts[2].split("=")[1]
].replace("L", "").trim());
                    double r = Double.parseDouble(parts[3].split("=")[1]
].replace("L", "").trim());
                    boiler.applyEnvironment(a, f, d, r);
                    boiler.tickSeconds(1.0);
                    Thread.sleep(delayMs);
                } catch (Throwable ignored) {

```

```

        }
    }
} catch (IOException e) {
    System.err.println("[Replay] Error: " + e.getMessage());
}
logger.log("Replay finished.");
}
}
}
/* ===== GUI ===== */
class BoilerGUI extends JFrame {
    private final SmartBoiler boiler;
    private final BDataLogger logger;
    private final BAlertSystem alerts;
    private final BoilerSimulationEngine engine;
    private final BoilerReplayEngine replay;
    private JLabel lblT, lblMin, lblMax, lblHeat, lblPress, lblFlow, lblAmb, lblLime, lblEnergy;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JTextArea txtLog, txtAlerts;
    private JSpinner spMin, spMax, spMaxKW, spRampKW;
    private JSpinner spdAmb, spdFlow, spDraw, spRefill, spDt, spDelay;
    private JCheckBox cbRandomize;
    private JButton btnStart, btnStop, btnReplay, btnCalib, btnAdd, btnSub, btnApplyTh, btnApplyHeater;
    private javax.swing.Timer uiTimer;
    BoilerGUI(SmartBoiler boiler, BDataLogger logger, BAlertSystem alerts,
              BoilerSimulationEngine engine, BoilerReplayEngine replay)
    {
        super("SmartBoiler - Heating & Safety Control");
        this.boiler = boiler;
        this.logger = logger;
        this.alerts = alerts;
        this.engine = engine;
        this.replay = replay;
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        buildUi();
        setSize(1220, 800);
        setLocationRelativeTo(null);
        uiTimer = new javax.swing.Timer(500, e -> refreshUi());
        uiTimer.start();
        boiler.addListener(this::refreshUi);
    }
}

```

```

private JPanel titled(String title, JComponent inner) {
    JPanel p = new JPanel(new BorderLayout());
    p.setBorder(BorderFactory.createTitledBorder(title));
    p.add(inner, BorderLayout.CENTER);
    return p;
}

private void buildUi() {
    JPanel root = new JPanel(new BorderLayout(10,10));
    root.setBorder(new EmptyBorder(10,10,10,10));
    setContentPane(root);
    /* LEFT: Status & Controls */
    JPanel left = new JPanel();
    left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
    JPanel status = new JPanel(new GridLayout(0,2,6,6));
    lblT = new JLabel("-");
    lblMin = new JLabel("-");
    lblMax = new JLabel("-");
    lblHeat = new JLabel("-");
    lblPress = new JLabel("-");
    lblFlow = new JLabel("-");
    lblAmb = new JLabel("-");
    lblLime = new JLabel("-");
    lblEnergy = new JLabel("-");
    status.add(new JLabel("Temp (°C):"));      status.add(lblT);
    status.add(new JLabel("Min (°C):"));      status.add(lblMin);
    status.add(new JLabel("Max (°C):"));      status.add(lblMax);
    status.add(new JLabel("Heating:"));       status.add(lblHeat);
    status.add(new JLabel("Pressure (bar):")); status.add(lblPress);
;

    status.add(new JLabel("Flow (L/s):"));    status.add(lblFlow);
    status.add(new JLabel("Ambient (°C):"));  status.add(lblAmb);
    status.add(new JLabel("Limescale:"));     status.add(lblLime);
    status.add(new JLabel("Energy (kWh):"));  status.add(lblEnergy);
;

    left.add(titled("Live Status", status));
    JPanel th = new JPanel(new GridLayout(0,2,6,6));
    spMin = new JSpinner(new SpinnerNumberModel(40.0, 0.0, 100.0, 1
.0));
    spMax = new JSpinner(new SpinnerNumberModel(80.0, 0.0, 100.0, 1
.0));
    btnApplyTh = new JButton("Apply Thresholds");
    btnApplyTh.addActionListener(e -> {
        double mn = ((Number)spMin.getValue()).doubleValue();
        double mx = ((Number)spMax.getValue()).doubleValue();

```

```

        boiler.setThresholds(mx, mn);
    });
    th.add(new JLabel("Min temp (°C)")); th.add(spMin);
    th.add(new JLabel("Max temp (°C)")); th.add(spMax);
    th.add(new JLabel(" ")); th.add(btnApplyTh);
    left.add(Box.createVerticalStrut(6));
    left.add(titled("Thresholds", th));
//    JPanel heaterCfg = new JPanel(new GridLayout(0,2,6,6));
//    spMaxKW = new JSpinner(new SpinnerNumberModel(12.0, 1.0, 50.0,
//    0.5));
//    spRampKW= new JSpinner(new SpinnerNumberModel(1.0, 0.2, 5.0, 0
//    .2));
//    btnApplyHeater = new JButton("Apply Heater Limits");
//    btnApplyHeater.addActionListener(e -> boiler.setHeaterLimits(
//        ((Number)spMaxKW.getValue()).doubleValue(),
//        ((Number)spRampKW.getValue()).doubleValue()
//    ));
//    heaterCfg.add(new JLabel("Max Power (kW)")); heaterCfg.add(spM
//    axKW);
//    heaterCfg.add(new JLabel("Ramp (kW/step)")); heaterCfg.add(spR
//    ampKW);
//    heaterCfg.add(new JLabel(" ")); heaterCfg.add(btnApplyHeater);
//    left.add(Box.createVerticalStrut(6));
//    left.add(titled("Heater Controller", heaterCfg));
//
//    JPanel ops = new JPanel(new GridLayout(0,2,6,6));
//    btnAdd = new JButton("Manual +Temp +5°C");
//    btnSub = new JButton("Manual -Temp -5°C");
//    btnCalib = new JButton("Calibrate Temp");
//    btnAdd.addActionListener(e -> boiler.updateTemperature(+5.0));
//    btnSub.addActionListener(e -> boiler.updateTemperature(-5.0));
//    btnCalib.addActionListener(e -> {
//        String s = JOptionPane.showInputDialog(this, "Observed temp
//        erature (°C):", BStr.fmt2(boiler.getCurrentTemperature()));
//        if (s == null) return;
//        try { boiler.calibrateTemperature(Double.parseDouble(s)); }
//        catch (NumberFormatException ex) { JOptionPane.showMessageDialog(this, "Invalid number."); }
//    });
//    ops.add(btnAdd); ops.add(btnSub);
//    ops.add(btnCalib); ops.add(new JLabel(" "));
//    left.add(Box.createVerticalStrut(6));
//    left.add(titled("Operations", ops));
//

```

```

//      JPanel sim = new JPanel(new GridLayout(0,2,6,6));
//      spdAmb = new JSpinner(new SpinnerNumberModel(+0.03, -
5.0, 5.0, 0.01));
//      spdFlow= new JSpinner(new SpinnerNumberModel(+0.01, 0.0, 2.0,
0.01));
//      spDraw = new JSpinner(new SpinnerNumberModel(0.5, 0.0, 5.0, 0.
1));
//      spRefill = new JSpinner(new SpinnerNumberModel(0.5, 0.0, 5.0,
0.1));
//      spDt   = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0
.1));
//      spDelay= new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10
));
//      cbRandomize = new JCheckBox("Randomize", true);
//      btnStart = new JButton("Start Simulation");
//      btnStop  = new JButton("Stop Simulation");
//      btnReplay= new JButton("Replay from Log");
//      btnStart.addActionListener(e -> engine.setParams(
//          ((Number)spdAmb.getValue()).doubleValue(),
//          ((Number)spdFlow.getValue()).doubleValue(),
//          ((Number)spDraw.getValue()).doubleValue(),
//          ((Number)spRefill.getValue()).doubleValue(),
//          ((Number)spDt.getValue()).doubleValue(),
//          ((Number)spDelay.getValue()).LongValue(),
//          cbRandomize.isSelected()
//      ));

JPanel heaterCfg = new JPanel(new GridLayout(0,2,6,6));
spMaxKW = new JSpinner(new SpinnerNumberModel(12.0, 1.0, 50.0,
0.5));
spRampKW= new JSpinner(new SpinnerNumberModel(1.0, 0.2, 5.0, 0.
2));
btnApplyHeater = new JButton("Apply Heater Limits");
btnApplyHeater.addActionListener(e -> boiler.setHeaterLimits(
    ((Number)spMaxKW.getValue()).doubleValue(),
    ((Number)spRampKW.getValue()).doubleValue()
));
heaterCfg.add(new JLabel("Max Power (kW)")); heaterCfg.add(spMa
xKW);
heaterCfg.add(new JLabel("Ramp (kW/step)")); heaterCfg.add(spRa
mpKW);
heaterCfg.add(new JLabel(" ")); heaterCfg.add(btnApplyHeater);
left.add(Box.createVerticalStrut(6));
left.add(titled("Heater Controller", heaterCfg));

```

```

JPanel ops = new JPanel(new GridLayout(0,2,6,6));
btnAdd = new JButton("Manual +Temp +5°C");
btnSub = new JButton("Manual -Temp -5°C");
btnCalib = new JButton("Calibrate Temp");
btnAdd.addActionListener(e -> boiler.updateTemperature(+5.0));
btnSub.addActionListener(e -> boiler.updateTemperature(-5.0));
btnCalib.addActionListener(e -> {
    String s = JOptionPane.showInputDialog(this, "Observed temperature (°C):", BStr.fmt2(boiler.getCurrentTemperature()));
    if (s == null) return;
    try { boiler.calibrateTemperature(Double.parseDouble(s)); }
    catch (NumberFormatException ex) { JOptionPane.showMessageDialog(this, "Invalid number."); }
});
ops.add(btnAdd); ops.add(btnSub);
ops.add(btnCalib); ops.add(new JLabel(" "));
left.add(Box.createVerticalStrut(6));
left.add(titled("Operations", ops));
JPanel sim = new JPanel(new GridLayout(0,2,6,6));
spdAmb = new JSpinner(new SpinnerNumberModel(+0.03, -5.0, 5.0, 0.01));
spdFlow= new JSpinner(new SpinnerNumberModel(+0.01, 0.0, 2.0, 0.01));
spDraw = new JSpinner(new SpinnerNumberModel(0.5, 0.0, 5.0, 0.1));
spRefill = new JSpinner(new SpinnerNumberModel(0.5, 0.0, 5.0, 0.1));
spDt = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0.1));
spDelay= new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10));

cbRandomize = new JCheckBox("Randomize", true);
btnStart = new JButton("Start Simulation");
btnStop = new JButton("Stop Simulation");
btnReplay= new JButton("Replay from Log");

btnStart.addActionListener(e -> engine.setParams(
    ((Number)spdAmb.getValue()).doubleValue(),
    ((Number)spdFlow.getValue()).doubleValue(),
    ((Number)spDraw.getValue()).doubleValue(),
    ((Number)spRefill.getValue()).doubleValue(),
    ((Number)spDt.getValue()).doubleValue(),
    ((Number)spDelay.getValue()).longValue(),
    cbRandomize.isSelected()

```

```

));

btnStart.addActionListener(e -> engine.start());
btnStop.addActionListener(e -> engine.stop());
btnReplay.addActionListener(e -> new Thread(() -> replay.replay
(logger.getFile(), 60), "Replay").start());
sim.add(new JLabel("ΔAmbient (°C/tick)")); sim.add(spdBamb);
sim.add(new JLabel("ΔFlow (L/s/tick)"));    sim.add(spDFlow);
sim.add(new JLabel("Draw per tick (L)"));    sim.add(spDraw);
sim.add(new JLabel("Refill per tick (L)"));  sim.add(spRefill);
sim.add(new JLabel("dt (s)"));               sim.add(spDt);
sim.add(new JLabel("Delay (ms)"));           sim.add(spDelay);
sim.add(new JLabel(" "));                   sim.add(cbRandomize);
sim.add(btnStart);                          sim.add(btnStop);
sim.add(new JLabel(" "));                   sim.add(btnReplay);
left.add(Box.createVerticalStrut(6));
left.add(titled("Simulation", sim));
getContentPane().add(left, BorderLayout.WEST);
/* CENTER: Sensors + Alerts */
JPanel center = new JPanel(new BorderLayout(6,6));
tblModel = new DefaultTableModel(new String[]{"Sensor", "Value",
"OK", "Status"}, 0) {
    @Override public boolean isCellEditable(int row, int column
) { return false; }
};
tblSensors = new JTable(tblModel);
JScrollPane spTbl = new JScrollPane(tblSensors);
center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
txtAlerts = new JTextArea(8, 60);
txtAlerts.setEditable(false);
center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
getContentPane().add(center, BorderLayout.CENTER);
/* RIGHT: Log tail */
txtLog = new JTextArea();
txtLog.setEditable(false);
JScrollPane spLog = new JScrollPane(txtLog);
getContentPane().add(titled("Log (tail)", spLog), BorderLayout.
EAST);
}

private void refreshUi() {
    lblT.setText(BStr.fmt2(boiler.getCurrentTemperature()));
    lblMin.setText(BStr.fmt2(boiler.getMinTemperature()));
    lblMax.setText(BStr.fmt2(boiler.getMaxTemperature()));

```

```

        lblHeat.setText(boiler.isHeatingActive() ? "ON" : "OFF");
        lblHeat.setForeground(boiler.isHeatingActive() ? new Color(0,12
0,0) : Color.RED);
        lblPress.setText(BStr.fmt2(boiler.getPressureBar()));
        lblFlow.setText(BStr.fmt2(boiler.getFlowRateLps()));
        lblAmb.setText(BStr.fmt2(boiler.getAmbientTemperature()));
        lblLime.setText(BStr.fmt2(boiler.getLimescaleIdx()));
        lblEnergy.setText(BStr.fmt2(boiler.getEnergyKWh()));
        tblModel.setRowCount(0);
        tblModel.addRow(new Object[]{
            boiler.getTempSensor().getName(),
            BStr.fmt2(boiler.getTempSensor().getValue()),
            boiler.getTempSensor().isOk(), boiler.getTempSensor().g
etStatus()
        });
        tblModel.addRow(new Object[]{
            boiler.getPressureSensor().getName(),
            BStr.fmt2(boiler.getPressureSensor().getValue()),
            boiler.getPressureSensor().isOk(), boiler.getPressureSe
nsor().getStatus()
        });
        tblModel.addRow(new Object[]{
            boiler.getFlowSensor().getName(),
            BStr.fmt2(boiler.getFlowSensor().getValue()),
            boiler.getFlowSensor().isOk(), boiler.getFlowSensor().g
etStatus()
        });
        tblModel.addRow(new Object[]{
            boiler.getEnergySensor().getName(),
            BStr.fmt2(boiler.getEnergySensor().getValue()),
            boiler.getEnergySensor().isOk(), boiler.getEnergySensor
().getStatus()
        });
        tblModel.addRow(new Object[]{
            boiler.getLimeSensor().getName(),
            BStr.fmt2(boiler.getLimeSensor().getValue()),
            boiler.getLimeSensor().isOk(), boiler.getLimeSensor().g
etStatus()
        });
        // alerts
        if (txtAlerts != null) {
            StringBuilder sb = new StringBuilder();
            for (BAAlertSystem.Alert a : alerts.recent(18)) sb.append(a)
.append("\n");

```

```

        txtAlerts.setText(sb.toString());
        txtAlerts.setCaretPosition(txtAlerts.getDocument().getLength()
h());
    }
    // log tail
    if (txtLog != null) {
        List<String> lines = logger.recent(220);
        StringBuilder sb = new StringBuilder();
        for (String s : lines) sb.append(s).append("\n");
        txtLog.setText(sb.toString());
        txtLog.setCaretPosition(txtLog.getDocument().getLength());
    }
}
}
/* ===== CLI Harness ===== */
class BoilerCLIHarness {
    private final SmartBoiler boiler;
    BoilerCLIHarness(SmartBoiler boiler) { this.boiler = boiler; }
    void runAll() {
        System.out.println("==== CLI Demo (VT) ====");
        vt();
        System.out.println("==== CLI Demo (FT) ====");
        ft();
        System.out.println("==== CLI Demo (Z3 mimic) ====");
        z3();
        System.out.println("==== CLI Demo (UVT) ====");
        uvt();
    }
    // VT: original vector to trigger <min / between / >max branches
    void vt() {
        boiler.updateTemperature(-
25.0); // 60 -> 35 (<40) => heatingActive=true
        boiler.updateTemperature(20.0); // 35 -> 55 (between) => keep s
tate (true)
        boiler.updateTemperature(30.0); // 55 -> 85 (>80) => heatingAct
ive=false
        boiler.updateTemperature(-
10.0); // 85 -> 75 (between) => keep state (false)
    }
    // FT: extreme numeric stress (as user's FT)
    void ft() {
        boiler.updateTemperature(-1.360606968);
        boiler.updateTemperature(0.0);
        boiler.updateTemperature(4.9E-324);
    }
}

```

```

        boiler.updateTemperature(-3.934312560);
    }
    // Z3-like: mirror VT for deterministic coverage
    void z3() {
        boiler.updateTemperature(-25.0);
        boiler.updateTemperature(20.0);
        boiler.updateTemperature(30.0);
        boiler.updateTemperature(-10.0);
    }
    // UVT: mirror VT again
    void uvt() {
        boiler.updateTemperature(-25.0);
        boiler.updateTemperature(20.0);
        boiler.updateTemperature(30.0);
        boiler.updateTemperature(-10.0);
    }
}
/* ===== Main ===== */
public class c131_SmartBoilerSystemEx {
    public static void main(String[] args) {
        //      boolean forceCli = Arrays.asList(args).contains("--cli");
        boolean forceCli = true;

        // Logger & Alerts
        BDataLogger logger = new BDataLogger("boiler_log.txt", 900);
        BAlertSystem alertSystem = new BAlertSystem(logger);
        // Core boiler with user's parameters
        SmartBoiler boiler = new SmartBoiler(80.0, 40.0, 60.0, logger,
alertSystem);
        // Engines
        BoilerSimulationEngine engine = new BoilerSimulationEngine(boil
er, logger);
        BoilerReplayEngine replay = new BoilerReplayEngine(boiler, logg
er);

        // ===== CLI path (sequential 4 groups; comment 3 lines to run
single group) =====
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            BoilerCLIHarness harness = new BoilerCLIHarness(boiler);
            //      harness.vt();    // ← keep to run VT
            harness.ft();    // ← comment out to skip FT
            //      harness.z3();    // ← comment out to skip Z3
            //      harness.uvt();    // ← comment out to skip UVT
            // small sim to exercise sensors & dynamics

```

```

        for (int i = 0; i < 20; i++) {
            boiler.applyEnvironment(+0.03, +0.01, 0.5, 0.5);
            boiler.tickSeconds(1.0);
            try { Thread.sleep(100L); } catch (Exception ignored) {
        }

        }
        logger.log("CLI demo finished.");
        System.out.println("CLI demo finished. Log written to: " +
logger.getFile().getAbsolutePath());
        return;
    }
    // ===== GUI path =====
    logger.log("Running in GUI mode.");
    SwingUtilities.invokeLater(() -> {
        BoilerGUI gui = new BoilerGUI(boiler, logger, alertSystem,
engine, replay);
        gui.setVisible(true);
    });
}
private static boolean headless() {
    try { return GraphicsEnvironment.isHeadless(); }
    catch (Throwable t) { return true; }
}
}
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c131_SmartBoilerSystemEx.java	42.1 %	48	66	114
> BAlertSystem	0.0 %	0	2	2
> BoilerReplayEngine	0.0 %	0	8	8
> BoilerSimulationEngine	0.0 %	0	8	8
> BDataLogger	25.0 %	2	6	8
> c131_SmartBoilerSystemEx	50.0 %	3	3	6
> EnergyMeterSensor	50.0 %	3	3	6
> FlowRateSensor	50.0 %	3	3	6
> LimescaleSensor	50.0 %	5	5	10
> PressureSensor	50.0 %	4	4	8
> WaterTempSensor	50.0 %	4	4	8
▼ SmartBoiler	76.9 %	20	6	26
▶ applyEnvironment(double, double, d	50.0 %	2	2	4
▶ emit()	50.0 %	1	1	2
▶ sampleSensors()	75.0 %	6	2	8
▶ tickSeconds(double)	75.0 %	3	1	4
▶ updateTemperature(double)	100.0 %	8	0	8
▶ SmartBoiler(double, double, double,		0	0	0
▶ buildContext()		0	0	0
> HeaterController	100.0 %	4	0	4

3. Vibration testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c131_SmartBoilerSystemEx.java	34.2 %	1,132	2,177	3,309
> BoilerContext	6.1 %	3	46	49
> BoilerReplayEngine	6.2 %	9	136	145
> BoilerSimulationEngine	15.4 %	41	225	266
> BoilerCLIHarness	24.2 %	23	72	95
> BStr	54.5 %	12	10	22
> BAlertSystem	55.9 %	33	26	59
> BDataLogger	57.9 %	70	51	121
> c131_SmartBoilerSystemEx	68.2 %	75	35	110
> BSensor	69.0 %	20	9	29
▼ SmartBoiler	76.7 %	516	157	673
emit()	52.9 %	9	8	17
sampleSensors()	79.5 %	70	18	88
tickSeconds(double)	96.9 %	63	2	65
SmartBoiler(double, double, double, d	100.0 %	87	0	87
applyEnvironment(double, double, d	100.0 %	105	0	105
buildContext()	100.0 %	47	0	47
updateTemperature(double)	100.0 %	135	0	135

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c131_SmartBoilerSystemEx.java	34.2 %	39	75	114
> BAlertSystem	0.0 %	0	2	2
> BoilerReplayEngine	0.0 %	0	8	8
> BoilerSimulationEngine	0.0 %	0	8	8
> BDataLogger	25.0 %	2	6	8
> c131_SmartBoilerSystemEx	50.0 %	3	3	6
> EnergyMeterSensor	50.0 %	3	3	6
> FlowRateSensor	50.0 %	3	3	6
> HeaterController	50.0 %	2	2	4
> LimescaleSensor	50.0 %	5	5	10
> PressureSensor	50.0 %	4	4	8
▼ SmartBoiler	50.0 %	13	13	26
applyEnvironment(double, double, d	50.0 %	2	2	4
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	4	4	8
tickSeconds(double)	50.0 %	2	2	4
updateTemperature(double)	50.0 %	4	4	8
SmartBoiler(double, double, double,		0	0	0
buildContext()		0	0	0

5. Fuzz testing instruction coverage: 90.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c131_SmartBoilerSystemEx.java	30.5 %	1,010	2,299	3,309
> BoilerContext	6.1 %	3	46	49
> BoilerReplayEngine	6.2 %	9	136	145
> BoilerSimulationEngine	15.4 %	41	225	266
> BAlertSystem	18.6 %	11	48	59
> BoilerCLIHarness	24.2 %	23	72	95
> BStr	54.5 %	12	10	22
> BDataLogger	57.9 %	70	51	121
> HeaterController	67.1 %	47	23	70
> c131_SmartBoilerSystemEx	68.2 %	75	35	110
> BSensor	69.0 %	20	9	29
▼ SmartBoiler	71.2 %	479	194	673
emit()	52.9 %	9	8	17
sampleSensors()	54.5 %	48	40	88
updateTemperature(double)	90.4 %	122	13	135
tickSeconds(double)	93.8 %	61	4	65
SmartBoiler(double, double, double,	100.0 %	87	0	87
applyEnvironment(double, double, d	100.0 %	105	0	105
buildContext()	100.0 %	47	0	47
> LimescaleSensor	89.7 %	52	6	58

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c131_SmartBoilerSystemEx.java	42.1 %	48	66	114
> BAlertSystem	0.0 %	0	2	2
> BoilerReplayEngine	0.0 %	0	8	8
> BoilerSimulationEngine	0.0 %	0	8	8
> BDataLogger	25.0 %	2	6	8
> c131_SmartBoilerSystemEx	50.0 %	3	3	6
> EnergyMeterSensor	50.0 %	3	3	6
> FlowRateSensor	50.0 %	3	3	6
> LimescaleSensor	50.0 %	5	5	10
> PressureSensor	50.0 %	4	4	8
> WaterTempSensor	50.0 %	4	4	8
▼ SmartBoiler	76.9 %	20	6	26
▶ applyEnvironment(double, double, d	50.0 %	2	2	4
▪ emit()	50.0 %	1	1	2
▪ sampleSensors()	75.0 %	6	2	8
▶ tickSeconds(double)	75.0 %	3	1	4
▶ updateTemperature(double)	100.0 %	8	0	8
▶ SmartBoiler(double, double, double,		0	0	0
▶ buildContext()		0	0	0
> HeaterController	100.0 %	4	0	4

7. Z3 instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c131_SmartBoilerSystemEx.java	34.2 %	1,132	2,177	3,309
> BoilerContext	6.1 %	3	46	49
> BoilerReplayEngine	6.2 %	9	136	145
> BoilerSimulationEngine	15.4 %	41	225	266
> BoilerCLIHarness	24.2 %	23	72	95
> BStr	54.5 %	12	10	22
> BAlertSystem	55.9 %	33	26	59
> BDataLogger	57.9 %	70	51	121
> c131_SmartBoilerSystemEx	68.2 %	75	35	110
> BSensor	69.0 %	20	9	29
▼ SmartBoiler	76.7 %	516	157	673
▪ emit()	52.9 %	9	8	17
▪ sampleSensors()	79.5 %	70	18	88
▶ tickSeconds(double)	96.9 %	63	2	65
▶ SmartBoiler(double, double, double,	100.0 %	87	0	87
▶ applyEnvironment(double, double, d	100.0 %	105	0	105
▶ buildContext()	100.0 %	47	0	47
▶ updateTemperature(double)	100.0 %	135	0	135
> HeaterController	77.1 %	54	16	70
> LimescaleSensor	89.7 %	52	6	58

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c131_SmartBoilerSystemEx.java	42.1 %	48	66	114
> BAlertSystem	0.0 %	0	2	2
> BoilerReplayEngine	0.0 %	0	8	8
> BoilerSimulationEngine	0.0 %	0	8	8
> BDataLogger	25.0 %	2	6	8
> c131_SmartBoilerSystemEx	50.0 %	3	3	6
> EnergyMeterSensor	50.0 %	3	3	6
> FlowRateSensor	50.0 %	3	3	6
> LimescaleSensor	50.0 %	5	5	10
> PressureSensor	50.0 %	4	4	8
> WaterTempSensor	50.0 %	4	4	8
▼ SmartBoiler	76.9 %	20	6	26
▶ applyEnvironment(double, double, d	50.0 %	2	2	4
▪ emit()	50.0 %	1	1	2
▪ sampleSensors()	75.0 %	6	2	8
▶ tickSeconds(double)	75.0 %	3	1	4
▶ updateTemperature(double)	100.0 %	8	0	8

9. Unimproved V-method instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c131_SmartBoilerSystemEx.java	34.2 %	1,132	2,177	3,309
> BoilerContext	6.1 %	3	46	49
> BoilerReplayEngine	6.2 %	9	136	145
> BoilerSimulationEngine	15.4 %	41	225	266
> BoilerCLIHarness	24.2 %	23	72	95
> BStr	54.5 %	12	10	22
> BAlertSystem	55.9 %	33	26	59
> BDataLogger	57.9 %	70	51	121
> c131_SmartBoilerSystemEx	68.2 %	75	35	110
> BSensor	69.0 %	20	9	29
▼ SmartBoiler	76.7 %	516	157	673
emit()	52.9 %	9	8	17
sampleSensors()	79.5 %	70	18	88
tickSeconds(double)	96.9 %	63	2	65
SmartBoiler(double, double, double, d	100.0 %	87	0	87
applyEnvironment(double, double, d	100.0 %	105	0	105
buildContext()	100.0 %	47	0	47
updateTemperature(double)	100.0 %	135	0	135
> HeaterController	77.1 %	54	16	70
> LimescaleSensor	89.7 %	52	6	58